# VM3 — NGINX Load Balancer Setup Guide

**Reverse Proxy & Load Balancer on Ubuntu Server 22.04 LTS**

---

## Variables — Adapt Before Use

```
LB_IP=<load-balancer-ip>              # e.g. 192.168.x.x
APP_SERVER_IP=<app-server-ip>         # e.g. 192.168.x.x
GATEWAY=<your-gateway-ip>             # e.g. 192.168.x.1
SUBNET=<your-subnet>                  # e.g. 192.168.x.0/24
SSH_KEY_PATH=<path-to-your-key>       # e.g. ~/.ssh/lb-vm
USER=<your-username>
```

---

## Table of Contents


1. VM Specifications
2. Ubuntu Server Installation
3. SSH Key Authentication
4. SSH Hardening
5. Firewall Configuration
6. System Updates
7. NGINX Installation
8. Load Balancer Configuration
9. Security Headers

10. Node Exporter Installation
11. Verification & Testing
12. Useful Commands

---

# 1. VM Specifications

## Recommended Specifications

| Parameter | Value | Notes |
|---|---|---|
| VM ID | 102 | Proxmox VM ID |
| Name | `lb-vm` | Hostname |
| CPU | 1 core, type: `host` | host type for best performance on single-node |
| RAM | 2048 MB during install | See installation note below |
| Disk | 20 GB (local-lvm, SCSI, iothread=on) | OS only, no data storage |
| Network | VirtIO, bridge: vmbr0, firewall: on | |
| OS | Ubuntu Server 22.04 LTS | |

## Network Configuration

| Field | Value |
|---|---|
| IPv4 Method | Manual |
| Subnet | `<SUBNET>` |

| Field | Value |
| --- | --- |
| Address | `<LB_IP>` |
| Gateway | `<GATEWAY>` |
| DNS | `1.1.1.1, 8.8.8.8` |

---

# 2. Ubuntu Server Installation

## Storage Layout

Accept the default LVM layout:

| Mount Point | Size | Type |
| --- | --- | --- |
| `/boot` | ~1.8 GB | ext4 |
| `/` | ~10 GB | ext4 (LVM) |
| *(free)* | ~8 GB | unallocated |

## Profile Configuration

| Field | Value |
| --- | --- |
| Your name | `<USER>` |
| Server name | `lb-vm` |
| Username | `<USER>` |
| Password | *(strong password)* |

## SSH & Snaps

- Install OpenSSH server: enabled
- Allow password authentication: enabled temporarily
- Featured snaps: skip all

---

# 3. SSH Key Authentication

## Reuse Existing Key

If you already have an SSH key from another VM in the same project, reuse it — copy the same public key.

## Generate New Key (if needed)

**Linux/macOS:**

```
ssh-keygen -t ed25519 -f ~/.ssh/lb-vm -C "lb-vm"
```

**Windows PowerShell:**

```
ssh-keygen -t ed25519 -f "$env:USERPROFILE\.ssh\lb-vm" -C "lb-vm"
```

## Copy Public Key to VM

**Linux/macOS:**

```
ssh-copy-id -i ~/.ssh/lb-vm.pub <USER>@<LB_IP>
```

**Windows PowerShell:**

```
type $env:USERPROFILE\.ssh\lb-vm.pub | ssh <USER>@<LB_IP> "mkdir -p
~/.ssh && chmod 700 ~/.ssh && cat >> ~/.ssh/authorized_keys && chmod 600
~/.ssh/authorized_keys"
```

### Verify Key

```
cat ~/.ssh/authorized_keys
```

### Test Key Login

#### Linux/macOS:

```
ssh -i ~/.ssh/lb-vm <USER>@<LB_IP>
```

#### Windows PowerShell:

```
ssh -i "$env:USERPROFILE\.ssh\lb-vm" <USER>@<LB_IP>
```

---

## 4. SSH Hardening

```
sudo sed -i 's/#\?PasswordAuthentication.*/PasswordAuthentication no/'
/etc/ssh/sshd_config
sudo sed -i 's/#\?PubkeyAuthentication.*/PubkeyAuthentication yes/'
/etc/ssh/sshd_config
sudo sed -i 's/#\?PermitRootLogin.*/PermitRootLogin no/'
/etc/ssh/sshd_config
```

### Verify

```
grep -E "PasswordAuthentication|PubkeyAuthentication|PermitRootLogin"
/etc/ssh/sshd_config
```

Expected output:

```
PasswordAuthentication no
PubkeyAuthentication yes
PermitRootLogin no
```

### Apply

```
sudo systemctl restart sshd
```

---

## 5. Firewall Configuration

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp      # SSH
sudo ufw allow 80/tcp      # HTTP
sudo ufw allow 443/tcp     # HTTPS
sudo ufw enable
```

### Verify

```
sudo ufw status
```

Expected output:

```
Status: active

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW       Anywhere
80/tcp                     ALLOW       Anywhere
443/tcp                    ALLOW       Anywhere
```

> *NOTE: Node Exporter port (9100) is restricted to monitoring VM only — see Section 10.*

---

# 6. System Updates

```
sudo apt update && sudo apt upgrade -y
sudo apt autoremove -y
```

---

# 7. NGINX Installation

### Install

```
sudo apt install nginx -y
```

### Enable & Start

```
sudo systemctl enable nginx
sudo systemctl start nginx
```

### Verify

```
sudo systemctl status nginx
```

Test default page in browser: `http://<LB_IP>` — should show the NGINX welcome page.

---

# 8. Load Balancer Configuration

### Create Config File

```
sudo nano /etc/nginx/sites-available/loadbalancer.conf
```

## Configuration

```
upstream app_servers {
    server <APP_SERVER_IP>:80;

    # Add more app servers as needed:
    # server <APP_SERVER_2_IP>:80;
}

server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://app_servers;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

## Enable Configuration

```
# Enable site
sudo ln -s /etc/nginx/sites-available/loadbalancer.conf /etc/nginx/sites-
enabled/

# Remove default site
sudo rm /etc/nginx/sites-enabled/default
```

## Validate & Reload

```
sudo nginx -t
sudo systemctl reload nginx
```

Expected validation output:

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

---

# 9. Security Headers

Add security headers to the NGINX config — required for A+ security rating:

```
sudo nano /etc/nginx/sites-available/loadbalancer.conf
```

Updated configuration with headers:

```
upstream app_servers {
    server <APP_SERVER_IP>:80;
}

server {
    listen 80;
    server_name _;

    # Security Headers
    add_header X-Frame-Options "DENY";
    add_header X-Content-Type-Options "nosniff";
    add_header X-XSS-Protection "1; mode=block";
    add_header Content-Security-Policy "default-src 'self'; style-src
'self';";
    add_header Referrer-Policy "strict-origin-when-cross-origin";
    add_header Strict-Transport-Security "max-age=31536000;
includeSubDomains" always;
    add_header Permissions-Policy "geolocation=(), microphone=(), camera=
()";

    location / {
        proxy_pass http://app_servers;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

## Header Reference

| Header | Value | Purpose |
|---|---|---|
| `X-Frame-Options` | `DENY` | Prevents clickjacking |
| `X-Content-Type-Options` | `nosniff` | Prevents MIME sniffing attacks |
| `X-XSS-Protection` | `1; mode=block` | Enables browser XSS filter |
| `Content-Security-Policy` | `default-src 'self'` | Restricts resources to same origin |
| `Referrer-Policy` | `strict-origin-when-cross-origin` | Controls referrer header |
| `Strict-Transport-Security` | `max-age=31536000; includeSubDomains` | Forces HTTPS for 1 year |
| `Permissions-Policy` | `geolocation=(), microphone=(), camera=()` | Disables browser features |

> *NOTE: `max-age=31536000` = 60 x 60 x 24 x 365 = exactly 1 year in seconds. Tells browsers to enforce HTTPS for all subsequent requests without attempting HTTP first.*

## Apply

```
sudo nginx -t && sudo systemctl reload nginx
```

## Verify Security Grade

Scan at `https://securityheaders.com` — target grade: **A+**

---

# 10. Node Exporter Installation

Node Exporter exposes system metrics (CPU, RAM, disk, network) to Prometheus on the monitoring VM.

## Install

```
sudo apt install prometheus-node-exporter -y
sudo systemctl enable prometheus-node-exporter
sudo systemctl start prometheus-node-exporter
```

## Restrict Port to Monitoring VM Only

```
sudo ufw allow from <MONITORING_IP> to any port 9100
```

## Verify

```
sudo systemctl status prometheus-node-exporter
```

## Add to Prometheus Config (on monitoring VM)

```
sudo nano /etc/prometheus/prometheus.yml
```

Add:

```
  - job_name: 'node-lb-vm'
    static_configs:
      - targets: ['<LB_IP>:9100']
```

Restart Prometheus:

```
sudo systemctl restart prometheus
```

---

# 11. Verification & Testing

## Services Summary

| Service | Port | Check Command |
|---|---|---|
| SSH | 22 | `sudo systemctl status sshd` |
| NGINX | 80 | `sudo systemctl status nginx` |
| Node Exporter | 9100 | `sudo systemctl status prometheus-node-exporter` |

## All Services at Once

```
for svc in sshd nginx prometheus-node-exporter; do
  echo "=== $svc ===" && sudo systemctl is-active $svc
done
```

## Test Connectivity to App VM

```
curl http://<APP_SERVER_IP>:80
```

## Test Load Balancer

```
curl http://<LB_IP>
```

## Network Traffic Flow

```
Internet / Cloudflare Tunnel
         ↓
NGINX lb-vm (<LB_IP>:80)
         ↓
upstream app_servers
         ↓
Flask app-vm (<APP_SERVER_IP>:80)
         ↓
PostgreSQL + Redis (Docker internal network)
```

---

## 12. Useful Commands

```
# Check NGINX status
sudo systemctl status nginx

# Test configuration syntax (always run before reload)
sudo nginx -t

# Reload config without downtime
sudo systemctl reload nginx

# Restart NGINX
sudo systemctl restart nginx

# View live access logs
sudo tail -f /var/log/nginx/access.log

# View live error logs
sudo tail -f /var/log/nginx/error.log

# List enabled sites
ls /etc/nginx/sites-enabled/

# View active config
cat /etc/nginx/sites-available/loadbalancer.conf
```

---

## Scaling — Adding More App Servers

```
sudo nano /etc/nginx/sites-available/loadbalancer.conf
```

```
upstream app_servers {
    server <APP_SERVER_1_IP>:80;
    server <APP_SERVER_2_IP>:80;   # new server
    server <APP_SERVER_3_IP>:80;   # another server
}
```

## Load Balancing Algorithms

```
# Round-Robin (default)
upstream app_servers {
    server <APP_SERVER_1_IP>:80;
    server <APP_SERVER_2_IP>:80;
}

# Least Connections
upstream app_servers {
    least_conn;
    server <APP_SERVER_1_IP>:80;
    server <APP_SERVER_2_IP>:80;
}

# IP Hash (session persistence)
upstream app_servers {
    ip_hash;
    server <APP_SERVER_1_IP>:80;
    server <APP_SERVER_2_IP>:80;
}

# Weighted
upstream app_servers {
    server <APP_SERVER_1_IP>:80 weight=3;
    server <APP_SERVER_2_IP>:80 weight=1;
}
```

Apply without downtime:

```
sudo nginx -t && sudo systemctl reload nginx
```

---

# Security Checklist

- [ ] SSH password authentication disabled
- [ ] Root SSH login disabled
- [ ] SSH key authentication enabled and tested
- [ ] UFW firewall active (ports 22, 80, 443 only)
- [ ] Node Exporter port (9100) restricted to monitoring VM only
- [ ] System packages up to date
- [ ] Default NGINX site removed
- [ ] NGINX config syntax validated before every reload
- [ ] Security headers configured (A+ on securityheaders.com)
- [ ] HSTS enabled (1 year, includeSubDomains)
- [ ] CSP enforced

---

# Troubleshooting

## 502 Bad Gateway

```
# Check upstream app is reachable
curl http://<APP_SERVER_IP>:80

# Check NGINX error log
sudo tail -f /var/log/nginx/error.log
```

NOTE: A 502 immediately after NGINX config is expected if the upstream app is not yet running.

## NGINX Config Error

```
sudo nginx -t
sudo journalctl -u nginx -n 50
```

## Installer Freeze at Profile Screen

Increase RAM to 2048 MB for installation only, then reduce back to 1024 MB after OS is installed. See Section 2.

## Config Changes Not Applied

```
# Always reload, not restart, for zero downtime
sudo nginx -t && sudo systemctl reload nginx
```